

TECH SCHOOL

The motivation of gRPC

Trong khóa học này, chúng ta sẽ tìm hiểu về gRPC và cách sử dụng gRPC kết hợp với Protocol Buffers để phát triển ứng dụng bằng Go và Java. Trước khi đi vào chi tiết kỹ thuật, điều quan trọng là phải hiểu được động lực ra đời của gRPC, hay nói cách khác, vấn đề mà gRPC được tạo ra để giải quyết.

Vấn đề cốt lõi mà gRPC hướng tới chính là giao tiếp giữa các ứng dụng. Trong thực tế, các ứng dụng hiện đại thường được xây dựng bằng nhiều ngôn ngữ lập trình khác nhau. Ví dụ, phần backend có thể được viết bằng Go, trong khi ứng dụng Android sử dụng Java và ứng dụng iOS sử dụng Swift. Khi đó, câu hỏi đặt ra là: làm thế nào để các hệ thống này có thể giao tiếp với nhau một cách hiệu quả?

Xu hướng phát triển phổ biến hiện nay là kiến trúc microservices. Trong kiến trúc này, ngay cả phía backend cũng có thể bao gồm rất nhiều dịch vụ nhỏ, mỗi dịch vụ được xây dựng bằng một ngôn ngữ khác nhau như Go, Python hay Rust, tùy thuộc vào yêu cầu nghiệp vụ và các ràng buộc kỹ thuật. Để các dịch vụ này có thể trao đổi dữ liệu với nhau, chúng cần thống nhất một tập hợp các hợp đồng API (API contracts).

Những hợp đồng API này không chỉ bao gồm kênh giao tiếp, mà còn liên quan đến cơ chế xác thực, định dạng dữ liệu truyền tải, mô hình dữ liệu, cũng như cách xử lý lỗi. Có rất nhiều yếu tố cần phải cân nhắc, và chính vì vậy, việc xây dựng một API hoàn chỉnh và chuẩn hóa là một công việc không hề đơn giản.

Bên cạnh đó, hiệu năng giao tiếp cũng là một yêu cầu rất quan trọng. Trong hệ thống microservices, số lượng thông điệp trao đổi giữa các dịch vụ là vô cùng lớn. Do đó, giao tiếp càng nhanh và càng nhẹ thì hệ thống càng hoạt động hiệu quả. Điều này càng trở nên quan trọng hơn trong các môi trường như ứng dụng di động, nơi băng thông mạng và tốc độ kết nối còn nhiều hạn chế. Khi đó, một giao thức truyền thông gọn nhẹ sẽ giúp cải thiện đáng kể trải nghiệm người dùng.

Ngoài hiệu năng, tính đơn giản trong giao tiếp cũng là một yếu tố then chốt. Hãy tưởng tượng một hệ thống có hàng trăm, thậm chí hàng nghìn microservices. Chúng ta chắc chắn không muốn dành phần lớn thời gian chỉ để viết mã phục vụ cho việc các dịch vụ “nói chuyện” với nhau. Điều mà các nhà phát triển mong muốn là một framework cho phép họ tập trung vào việc xây dựng logic nghiệp vụ cốt lõi, trong khi các vấn đề như giao tiếp, tuần tự hóa dữ liệu, và xử lý lỗi được framework đảm nhiệm.

Và framework đó chính là gRPC. gRPC được thiết kế để giải quyết toàn bộ những thách thức nêu trên: chuẩn hóa giao tiếp giữa các dịch vụ đa ngôn ngữ, đảm bảo hiệu năng cao, giao thức nhẹ, và đơn giản hóa quá trình phát triển hệ thống phân tán.

Qua phần này, chúng ta đã có cái nhìn tổng quan về những vấn đề mà gRPC hướng tới giải quyết. Trong bài học tiếp theo, chúng ta sẽ tìm hiểu chi tiết hơn về gRPC là gì và cách gRPC hoạt động để đạt được những mục tiêu đó.

What is gRPC and how it works

Trong bài học trước, chúng ta đã tìm hiểu rằng gRPC được tạo ra nhằm giải quyết bài toán giao tiếp giữa các ứng dụng và dịch vụ. Dựa trên nền tảng đó, phần này sẽ đi sâu hơn vào cách gRPC hoạt động và bản chất của gRPC là gì.

gRPC là một framework mã nguồn mở, có hiệu năng cao và được trang bị nhiều tính năng mạnh mẽ để xây dựng hệ thống giao tiếp giữa các dịch vụ. Framework này ban đầu được phát triển bởi Google và hiện nay đã trở thành một dự án thuộc Cloud Native Computing Foundation (CNCF), tương tự như Kubernetes hay Prometheus. Điều này cho thấy gRPC đã được cộng đồng công nghệ đón nhận rộng rãi và đóng vai trò quan trọng trong hệ sinh thái cloud-native.

Tên gọi gRPC cũng mang một ý nghĩa thú vị. Chữ “G” trong gRPC không cố định cho một từ duy nhất. Trong từng phiên bản phát hành khác nhau, chữ “G” có thể đại diện cho nhiều ý nghĩa khác nhau như good, green, glorious, hoặc thậm chí là chính gRPC. Điều này phản ánh tinh thần phát triển linh hoạt và vui nhộn của dự án.

Phần “RPC” trong gRPC là viết tắt của Remote Procedure Call. Đây là một giao thức cho phép một chương trình gọi và thực thi một thủ tục (procedure) của một chương trình khác đang chạy trên một máy tính khác. Điểm nổi bật của RPC là lập trình viên không cần phải tự tay xử lý các chi tiết phức tạp liên quan đến mạng, vì toàn bộ quá trình này đã được framework bên dưới tự động đảm nhiệm.

Nhờ cơ chế RPC, trong mã nguồn phía client, việc gọi một hàm trên server trông giống như gọi một hàm cục bộ thông thường. Cách tiếp cận này giúp đơn giản hóa đáng kể quá trình phát triển, ngay cả khi client và server được viết bằng các ngôn ngữ lập trình khác nhau, chẳng hạn như client viết bằng Go trong khi server được viết bằng Rust.

Để làm được điều đó, gRPC sử dụng khái niệm stub. Ở phía client, stub cung cấp các phương thức có chữ ký giống hệt như các hàm được định nghĩa ở phía server. Các stub này không do lập trình viên viết thủ công, mà được gRPC tự động sinh ra. Khi client gọi một phương thức trên stub, stub sẽ gọi gRPC framework để thực hiện việc trao đổi dữ liệu với server thông qua mạng.

Nhờ cơ chế stub này, client và server chỉ cần tập trung vào việc triển khai logic nghiệp vụ cốt lõi của dịch vụ, trong khi toàn bộ các vấn đề liên quan đến giao tiếp mạng, tuần tự hóa dữ liệu và truyền nhận thông điệp đều được gRPC xử lý một cách trong suốt. Trong bài học tiếp theo, chúng ta sẽ tìm hiểu cách gRPC tạo ra các stub này với sự hỗ trợ của Protocol Buffers.

gRPC code generation with Protocol Buffer

Sinh mã tự động (code generation) là một trong những tính năng quan trọng nhất của gRPC. Trong phần này, chúng ta sẽ tìm hiểu cách gRPC thực hiện việc sinh mã thông qua Protocol Buffers.

Để có thể tạo ra các stub cho cả server và client, bước đầu tiên là xây dựng hợp đồng API. Hợp đồng này bao gồm mô tả các dịch vụ (services) và các thông điệp dữ liệu (payload messages), được định nghĩa trong một file Protocol Buffer (file .proto). Trong file này, chúng ta khai báo các phương thức mà service cung cấp, cùng với kiểu dữ liệu đầu vào và đầu ra của chúng.

Ví dụ, một phương thức có tên là Hello được định nghĩa để nhận vào một thông điệp HelloRequest và trả về một thông điệp HelloResponse. Trong đó, HelloRequest chỉ chứa một trường kiểu chuỗi là name, còn HelloResponse chứa một trường kiểu chuỗi là greet. Đây là một ví dụ rất đơn giản, nhưng đủ để minh họa cách gRPC mô tả API thông qua Protocol Buffers. Các chi tiết nâng cao hơn sẽ được trình bày trong phần thực hành chuyên sâu về Protocol Buffers ở các bài học sau.

Từ file .proto này, mã stub cho server và client sẽ được tự động sinh ra bởi trình biên dịch Protocol Buffer, còn được gọi là protoc. Tùy theo ngôn ngữ lập trình sử dụng, chúng ta cần chỉ định cho trình biên dịch plugin gRPC tương ứng. Kết quả là các đoạn mã stub được tạo ra cho từng ngôn ngữ, chẳng hạn như Go hay Rust, với cấu trúc phù hợp để tích hợp trực tiếp vào ứng dụng.

Một câu hỏi thường được đặt ra là tại sao gRPC lại sử dụng Protocol Buffers. Có nhiều lý do cho lựa chọn này. Trước hết, cú pháp của Protocol Buffers rất dễ đọc và dễ hiểu. Bên cạnh đó, nó hỗ trợ khả năng tương tác đa ngôn ngữ với cơ chế sinh mã tự động cho nhiều ngôn ngữ lập trình khác nhau. Dữ liệu được biểu diễn dưới dạng nhị phân, giúp giảm kích thước thông điệp, tăng tốc độ truyền tải và hiệu quả hơn trong việc tuần tự hóa so với các định dạng văn bản như JSON hay XML.

Ngoài ra, Protocol Buffers cung cấp một hợp đồng API có kiểu dữ liệu mạnh (strongly-typed) giữa client và server, giúp giảm thiểu lỗi và tăng độ an toàn trong quá trình phát triển. Nó cũng đưa ra một tập hợp các quy tắc chặt chẽ cho việc tiến hóa API, nhằm đảm bảo khả năng tương thích ngược và tương thích xuôi khi hệ thống được mở rộng hoặc nâng cấp.

Mặc dù Protocol Buffers là lựa chọn phổ biến nhất, nhưng nó không phải là lựa chọn bắt buộc khi làm việc với gRPC. Ngoài Protocol Buffers, gRPC còn có thể sử dụng các công nghệ khác như Google FlatBuffers hoặc Microsoft Bond. Tuy nhiên, Protocol Buffers vẫn được xem là một lựa chọn rất hoàn thiện và đáng tin cậy.

Protocol Buffers hỗ trợ nhiều ngôn ngữ lập trình, trong đó có 10 ngôn ngữ được hỗ trợ chính thức. Một số ngôn ngữ như Go, Java và NodeJS được triển khai thuần, trong khi các ngôn ngữ khác như C++, C#, Objective-C, Python, Ruby, Dart và PHP được xây dựng dựa trên gRPC Core viết bằng C. Ngoài ra, cộng đồng còn phát triển nhiều thư viện không chính thức cho các ngôn ngữ khác như Swift, Rust, TypeScript, Haskell và nhiều ngôn ngữ khác nữa.

Qua phần này, chúng ta đã hiểu được cách gRPC sử dụng Protocol Buffers để tự động sinh mã cho client và server. Trong bài học tiếp theo, chúng ta sẽ khám phá một “vũ khí bí mật” khác

của gRPC, đó chính là HTTP/2, nền tảng quan trọng giúp gRPC đạt được hiệu năng cao trong giao tiếp mạng.

HTTP-2 - The secret weapon of gRPC

Trong bài học này, chúng ta tìm hiểu về HTTP/2, giao thức truyền tải mà gRPC sử dụng, cũng như những đặc điểm khiến HTTP/2 trở thành một lựa chọn vượt trội cho các hệ thống giao tiếp hiệu năng cao.

gRPC sử dụng HTTP/2 làm giao thức truyền tải (transfer protocol). Nhờ đó, gRPC kế thừa nhiều tính năng mạnh mẽ mà HTTP/2 cung cấp. Một trong những đặc điểm quan trọng nhất là binary framing. Khác với các giao thức dựa trên văn bản, binary framing giúp việc truyền tải dữ liệu đạt hiệu năng cao hơn, ổn định hơn, nhẹ hơn về mặt kích thước và an toàn hơn trong quá trình giải mã. Chính vì sử dụng định dạng nhị phân, HTTP/2 kết hợp rất tốt với Protocol Buffers, vốn cũng biểu diễn dữ liệu dưới dạng nhị phân.

Bên cạnh đó, HTTP/2 sử dụng cơ chế nén header bằng HPACK, giúp giảm đáng kể chi phí overhead trong mỗi request và response. Việc nén header này góp phần cải thiện hiệu năng tổng thể của hệ thống, đặc biệt trong các môi trường có nhiều lời gọi dịch vụ liên tiếp như kiến trúc microservices.

Một tính năng nổi bật khác của HTTP/2 là multiplexing. Với multiplexing, client và server có thể gửi và nhận nhiều request và response song song trên một kết nối TCP duy nhất. Điều này giúp giảm độ trễ, tránh tình trạng nghẽn cổ chai do phải mở nhiều kết nối TCP, đồng thời cải thiện việc sử dụng tài nguyên mạng.

HTTP/2 còn hỗ trợ server push, cho phép server gửi về nhiều response chỉ với một request duy nhất từ client. Tính năng này đặc biệt hữu ích trong các trường hợp server biết trước những tài nguyên mà client sẽ cần, từ đó có thể chủ động gửi dữ liệu trước khi client yêu cầu. Nhờ vậy, độ trễ do nhiều vòng trao đổi (round-trip latency) giữa client và server được giảm thiểu đáng kể.

Các thử nghiệm thực tế cho thấy HTTP/2 mang lại hiệu năng vượt trội so với HTTP/1.1. Trong một ví dụ minh họa, khi tải 200 hình ảnh nhỏ từ server, HTTP/1.1 mất khoảng 1,26 giây để hoàn thành, trong khi HTTP/2 chỉ mất khoảng 0,62 giây. Kết quả này cho thấy HTTP/2 có thể nhanh gần gấp đôi so với HTTP/1.1 trong các tình huống tương tự.

Về mặt kiến trúc, HTTP/2 hoạt động dựa trên một kết nối TCP duy nhất, bên trong đó tồn tại nhiều stream hai chiều. Mỗi stream có một định danh riêng và chứa nhiều thông điệp hai chiều. Mỗi thông điệp, dù là request hay response, đều được chia nhỏ thành nhiều frame nhị phân. Frame là đơn vị nhỏ nhất trong HTTP/2, dùng để mang các loại dữ liệu khác nhau như header, data, setting hay priority. Trên thực tế, các stream không truyền riêng rẽ mà các frame của chúng được xen kẽ (interleaved) trên cùng một kết nối và sẽ được ghép lại đúng thứ tự khi đến phía bên kia. Chính lớp binary framing này là nền tảng giúp HTTP/2 hỗ trợ multiplexing hiệu quả.

Khi so sánh HTTP/2 với HTTP/1.1, có thể thấy nhiều khác biệt quan trọng. HTTP/2 là một giao thức nhị phân, trong khi HTTP/1.1 dựa trên văn bản. Header trong HTTP/2 được nén, còn trong HTTP/1.1 thì không. HTTP/2 hỗ trợ multiplexing, cho phép gửi nhiều request và response song song trên một kết nối, trong khi HTTP/1.1 chỉ xử lý từng request một, buộc phải mở nhiều kết nối TCP riêng biệt để đạt hiệu năng cao hơn. Ngoài ra, HTTP/2 hỗ trợ server push, một tính năng không tồn tại trong HTTP/1.1.

Về mặt lịch sử, HTTP/1.1 được phát hành từ năm 1997, trong khi HTTP/2 chỉ mới được chuẩn hóa và đưa vào sử dụng từ năm 2015. Sự khác biệt này phản ánh rõ ràng mục tiêu của HTTP/2 trong việc đáp ứng các yêu cầu hiện đại về hiệu năng và khả năng mở rộng của các hệ thống phân tán.

Kết thúc bài học này, chúng ta đã hiểu được vai trò của HTTP/2 và lý do vì sao nó là nền tảng quan trọng giúp gRPC đạt được hiệu năng cao. Trong bài học tiếp theo, chúng ta sẽ tìm hiểu về các trường hợp sử dụng của gRPC và so sánh gRPC với REST để thấy rõ hơn ưu và nhược điểm của từng cách tiếp cận.

Is gRPC better than REST

Trong bài học này, chúng ta sẽ tìm hiểu các trường hợp sử dụng của gRPC và so sánh gRPC với REST, hai cách tiếp cận phổ biến trong việc xây dựng API và giao tiếp giữa các hệ thống phân tán.

gRPC hỗ trợ bốn kiểu giao tiếp khác nhau, đáp ứng nhiều nhu cầu sử dụng trong thực tế. Kiểu đơn giản nhất là Unary RPC, trong đó client gửi một request duy nhất và server trả về một response duy nhất. Cách giao tiếp này khá tương đồng với các API HTTP truyền thống.

Kiểu thứ hai là Client Streaming RPC. Trong mô hình này, client gửi một luồng (stream) gồm nhiều thông điệp liên tiếp lên server, sau khi gửi xong toàn bộ dữ liệu, client sẽ nhận về một response duy nhất từ server. Kiểu giao tiếp này thường được sử dụng khi client cần gửi một lượng lớn dữ liệu theo từng phần nhỏ.

Tiếp theo là Server Streaming RPC, nơi client chỉ gửi một request duy nhất, nhưng server sẽ phản hồi bằng một luồng nhiều thông điệp. Cách tiếp cận này phù hợp với các tình huống server cần trả về dữ liệu liên tục hoặc một tập hợp kết quả được xử lý theo thời gian.

Phức tạp nhất là Bidirectional Streaming RPC. Với mô hình này, cả client và server đều có thể gửi và nhận nhiều thông điệp song song, theo thứ tự tùy ý và không bị chặn lẫn nhau. Hai phía không cần phải chờ phản hồi trước khi gửi tiếp thông điệp mới, mang lại sự linh hoạt rất cao cho các kịch bản giao tiếp thời gian thực.

Bốn kiểu giao tiếp trên cung cấp một cái nhìn tổng quan về cách gRPC hỗ trợ các mô hình truyền thông khác nhau. Trong các bài thực hành sau, chúng ta sẽ triển khai từng kiểu một để hiểu sâu hơn cách chúng hoạt động trong thực tế.

Khi so sánh gRPC với REST, có thể nhận thấy nhiều khác biệt quan trọng. Trước hết, gRPC sử dụng HTTP/2, vốn nhanh và hiệu quả hơn so với HTTP/1.1 thường được dùng trong REST. Về định dạng dữ liệu, gRPC sử dụng Protocol Buffers để tuần tự hóa dữ liệu dưới dạng nhị phân, giúp thông điệp nhỏ gọn và hiệu quả hơn so với JSON, định dạng văn bản phổ biến trong REST.

Một điểm khác biệt quan trọng nữa là hợp đồng API. Trong gRPC, hợp đồng API là bắt buộc và được định nghĩa rõ ràng trong file .proto. Ngược lại, trong REST, hợp đồng API thường mang tính linh hoạt và không bắt buộc; nếu cần, có thể mô tả bằng OpenAPI hoặc Swagger, nhưng đây không phải là yêu cầu bắt buộc.

Về mặt sinh mã, gRPC cung cấp code generation tích hợp sẵn thông qua trình biên dịch Protocol Buffers. Trong khi đó, REST thường phải dựa vào các công cụ bên thứ ba như OpenAPI hay Swagger để sinh mã client và server. Cả gRPC và REST đều hỗ trợ bảo mật thông qua SSL/TLS.

gRPC hỗ trợ streaming hai chiều, trong khi REST chủ yếu chỉ hỗ trợ mô hình request-response một chiều từ client đến server. Nhờ những đặc điểm này, gRPC vượt trội hơn REST trong nhiều khía cạnh liên quan đến hiệu năng và khả năng giao tiếp linh hoạt.

Tuy nhiên, REST vẫn có một lợi thế quan trọng, đó là khả năng hỗ trợ trình duyệt. REST được hỗ trợ đầy đủ trên hầu hết các trình duyệt web, trong khi gRPC có mức độ hỗ trợ hạn chế và thường cần sử dụng gRPC-Web kết hợp với một lớp proxy để chuyển đổi giữa HTTP/1.1 và HTTP/2.

Do đó, gRPC có nhiều điểm mạnh nhưng cũng tồn tại những hạn chế nhất định. Để tận dụng tối đa lợi ích của gRPC, cần lựa chọn đúng bối cảnh sử dụng. gRPC đặc biệt phù hợp với kiến trúc microservices, nơi yêu cầu độ trễ thấp, thông lượng cao và hợp đồng API chặt chẽ. Nó cũng rất thích hợp cho môi trường đa ngôn ngữ (polyglot) nhờ khả năng sinh mã sẵn có cho nhiều ngôn ngữ lập trình.

Ngoài ra, gRPC là lựa chọn lý tưởng cho các hệ thống giao tiếp thời gian thực point-to-point, nhờ khả năng hỗ trợ streaming hai chiều hiệu quả. Cuối cùng, gRPC cũng rất phù hợp với các môi trường hạn chế về mạng, chẳng hạn như ứng dụng di động, do định dạng thông điệp gọn nhẹ và hiệu quả.

Kết thúc phần này, chúng ta đã hoàn thành toàn bộ các bài giảng lý thuyết của khóa học gRPC. Trong các bài học tiếp theo, chúng ta sẽ chuyển sang phần thực hành, nơi các khái niệm đã học sẽ được áp dụng trực tiếp vào việc xây dựng ứng dụng gRPC.